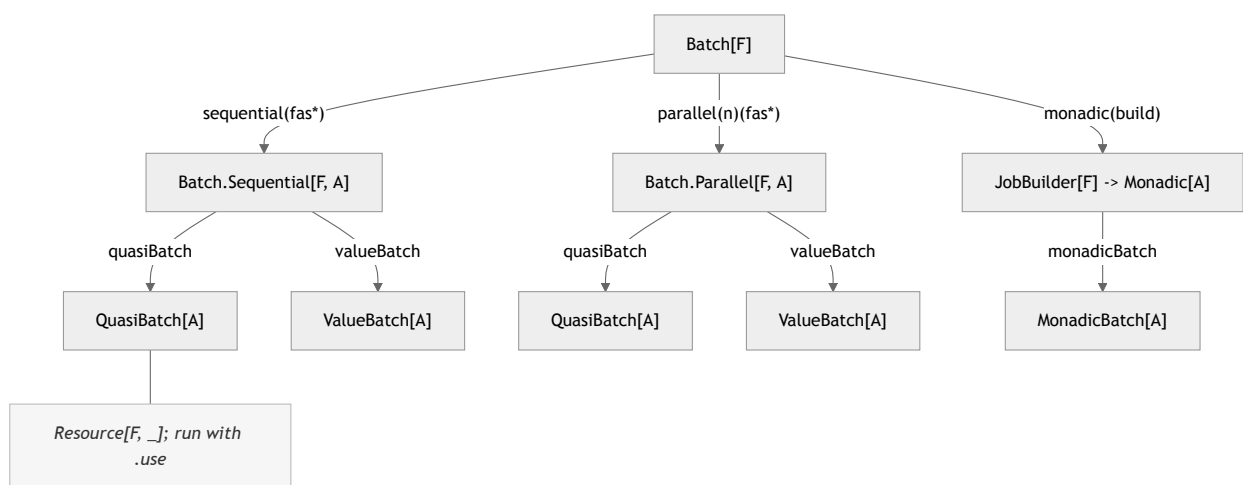


Batch execution

The `batch` package runs a group of named jobs under a shared metric scope, records per-job timing and outcomes, and produces a single aggregate `BatchResult`. This document describes how the pieces fit together.

Entry points

`Batch[F]` is the façade. From it you pick one of three execution shapes, then choose how failures are handled.

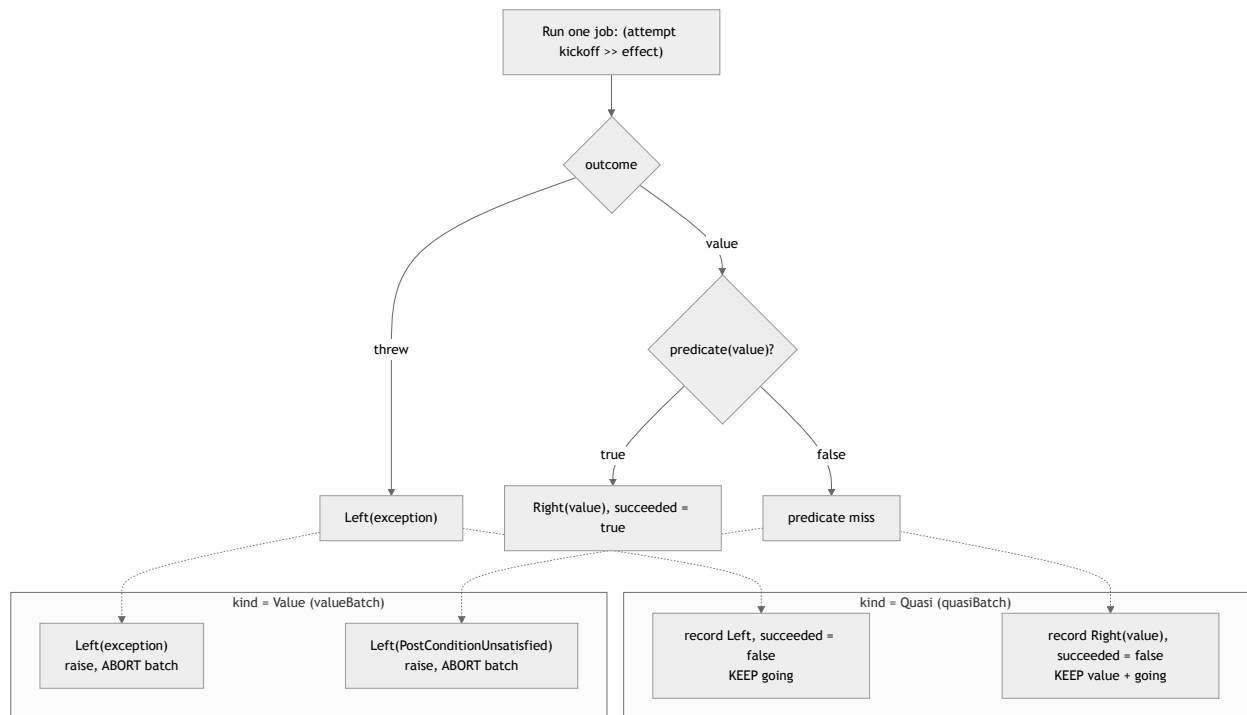


- **Sequential / Parallel** — independent jobs. `quasiBatch` collects every outcome (failures included); `valueBatch` propagates the first failure and keeps only successful values.
- **Monadic** — later jobs depend on earlier results, composed with `map` / `flatMap`. Produces a `MonadicBatch` carrying the step history and the final `Either`.

All of `quasiBatch`, `valueBatch`, and `monadicBatch` return a `Resource[F, _]`; the batch runs when the resource is used, and the active gauge / writers are released on close.

The two failure models

The `BatchKind` on each job decides what a failure means. This is the core distinction between `quasiBatch` and `valueBatch`.

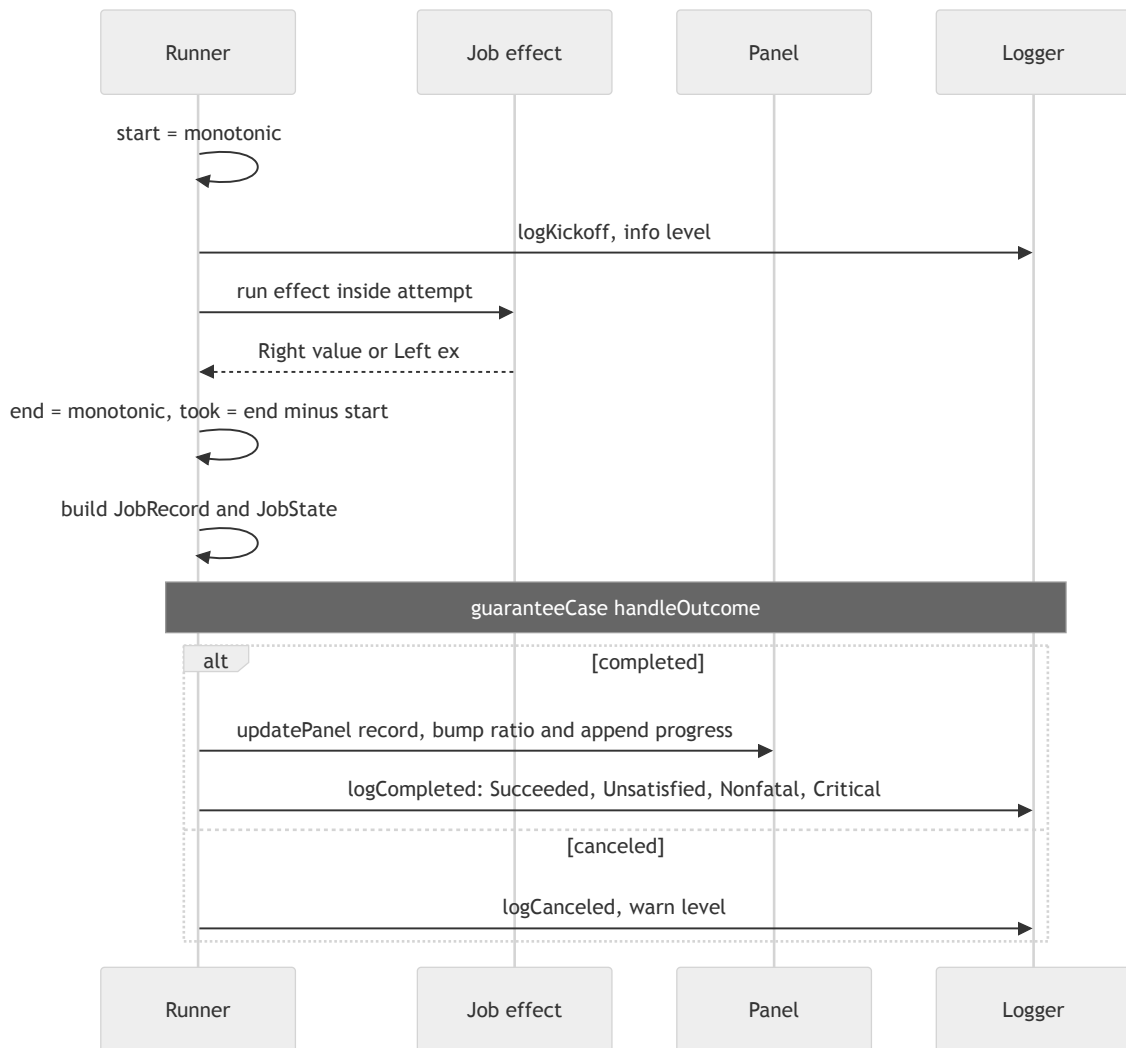


So:

- **Quasi** always runs to completion. `succeeded` on the result is always `true` ; `allPassed` is `false` if any job threw or was rejected by its predicate.
- **Value** stops at the first failing or rejected job by raising, so a `ValueBatch` only ever exists when every retained job succeeded (`succeeded` and `allPassed` are both `true`).

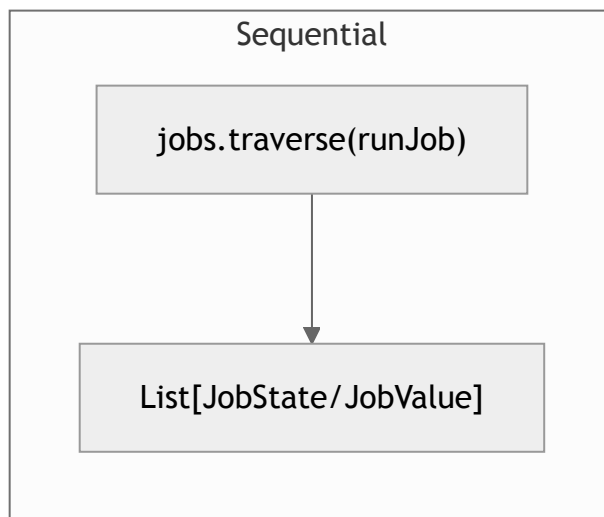
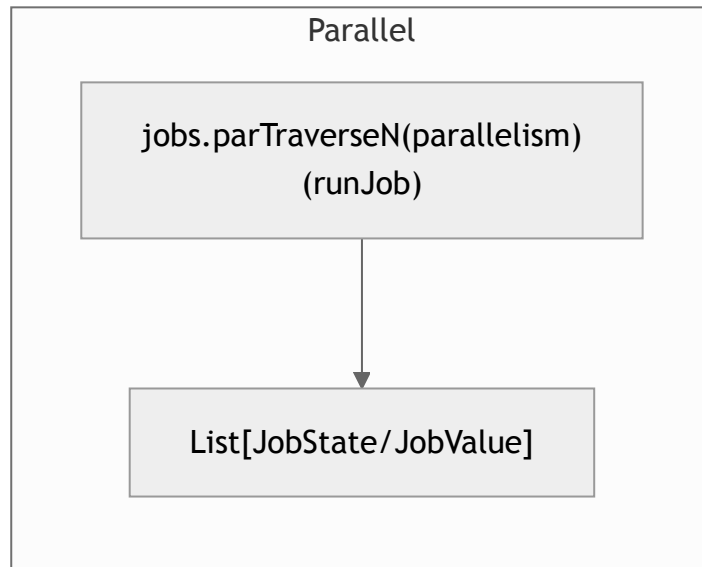
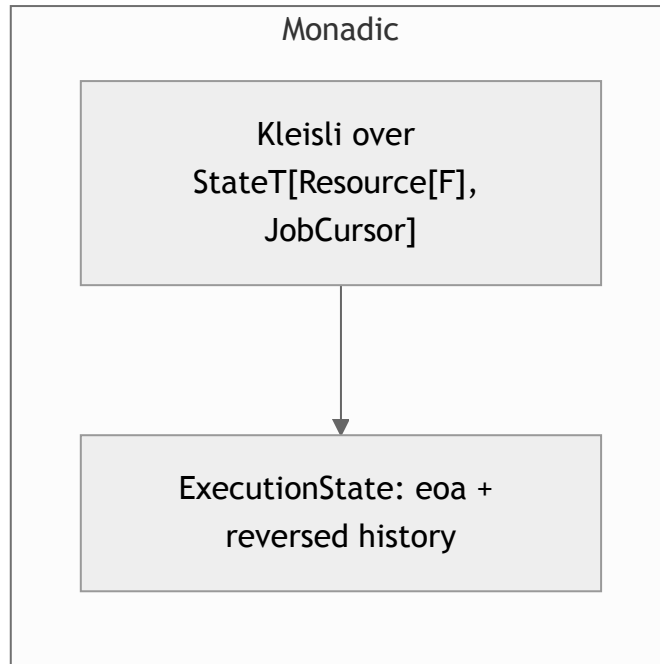
Per-job lifecycle

Every tracked job (sequential, parallel, or monadic) runs through the same steps. Timing brackets the kickoff log plus the effect; the completion log is written after `end` and is not part of `took` .



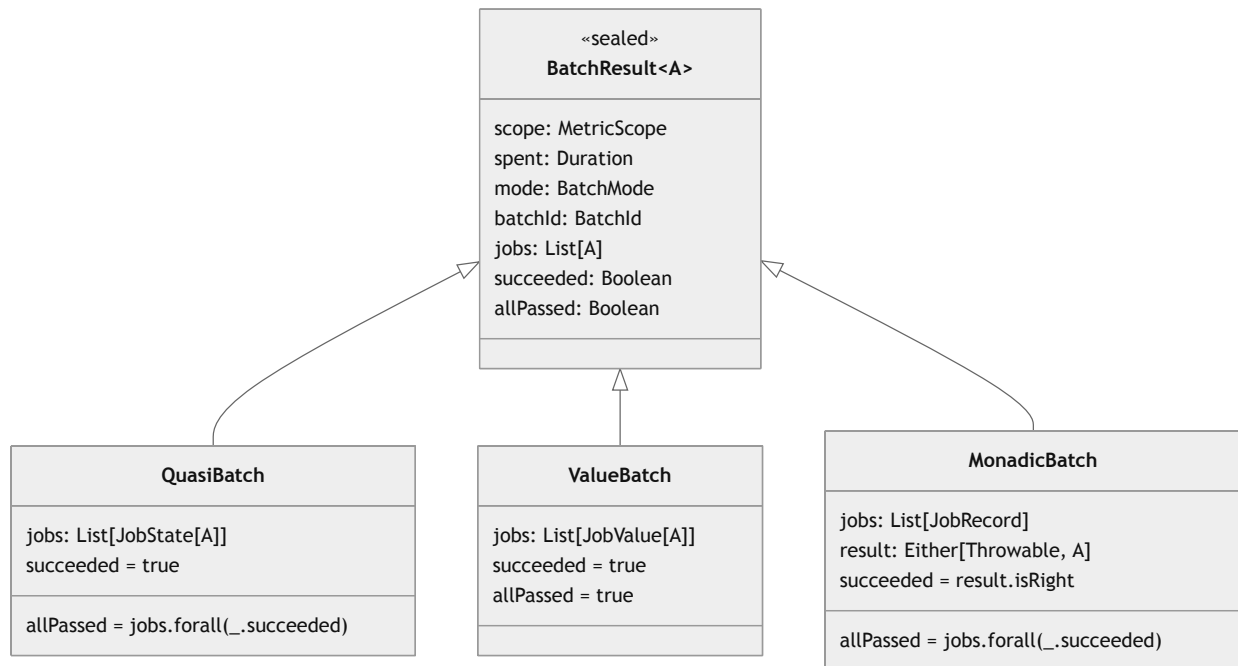
`handleOutcome` runs in a finalizer (`guaranteeCase`), so progress and completion logging happen even on cancellation. `Outcome.Errorred` is treated as impossible because the kickoff and effect are wrapped in `attempt`.

How the three modes differ



- **Sequential** threads jobs with `traverse` ; **Parallel** with `parTraverseN` . Both time the whole traversal (`spent`) and build the result from the collected `JobState` / `JobValue` list.
- **Monadic** threads a `JobCursor` (running index + carry-over start time) through a `StateT / Kleisli` , so each job's `start` is the previous job's `end` . Its `took` therefore absorbs any invisible `untracked` / `pure` steps between jobs, and the per-job durations sum exactly to `spent` . A `Left` short-circuits the remaining chain; `withFilter` turns a rejected value into a `PostConditionUnsatisfied Left` .

Result types



- `succeeded` — did the batch operation itself complete? Quasi and Value always do; Monadic completes only when its chain is not short-circuited.
- `allPassed` — did every job satisfy its post-condition? Can be `false` even when `succeeded` is `true` (a completed quasi/monadic batch with some rejected jobs).

See `../src/main/scala/com/github/chenharryhua/nanjin/guard/batch/data.scala` for the result and job types, and `internal.scala` for `ExecutionState` , `JobCursor` , and the log-entry classification.

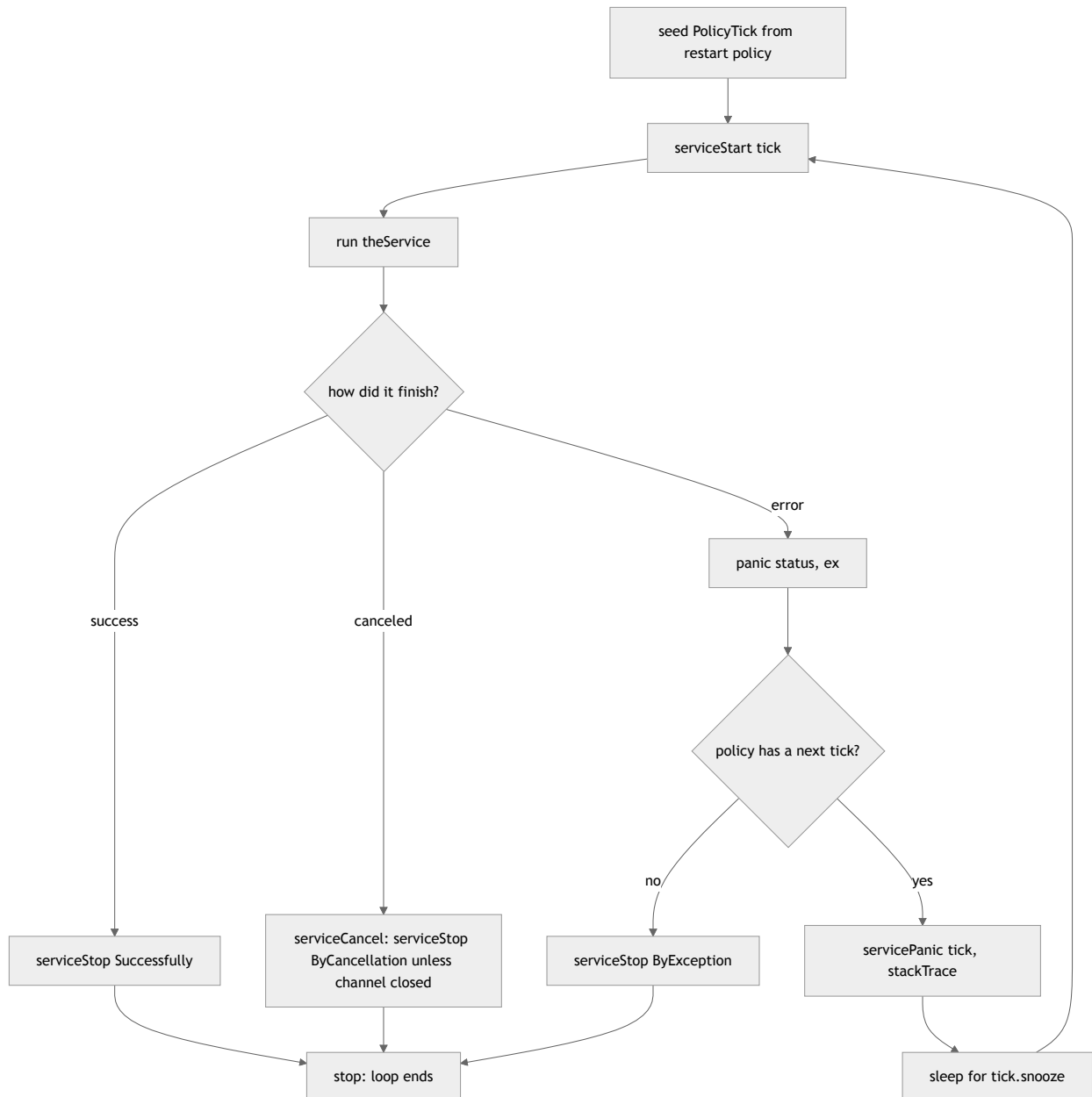
Watchdog

The watchdog is the supervisor that keeps a service running. It runs the user's service effect, and when that effect fails it decides — using the configured restart policy — whether to wait and restart, or to give up and stop. It also emits the service lifecycle events (`ServiceStart` , `ServicePanic` , `ServiceStop`) that observers see.

`watchdog(theService, handler)` returns a `Stream[F, Nothing]` that `ServiceGuard` runs concurrently with the main event channel, alongside periodic metric reporting and the HTTP server. `theService` is the deferred agent work; `handler` publishes lifecycle events.

Supervision loop

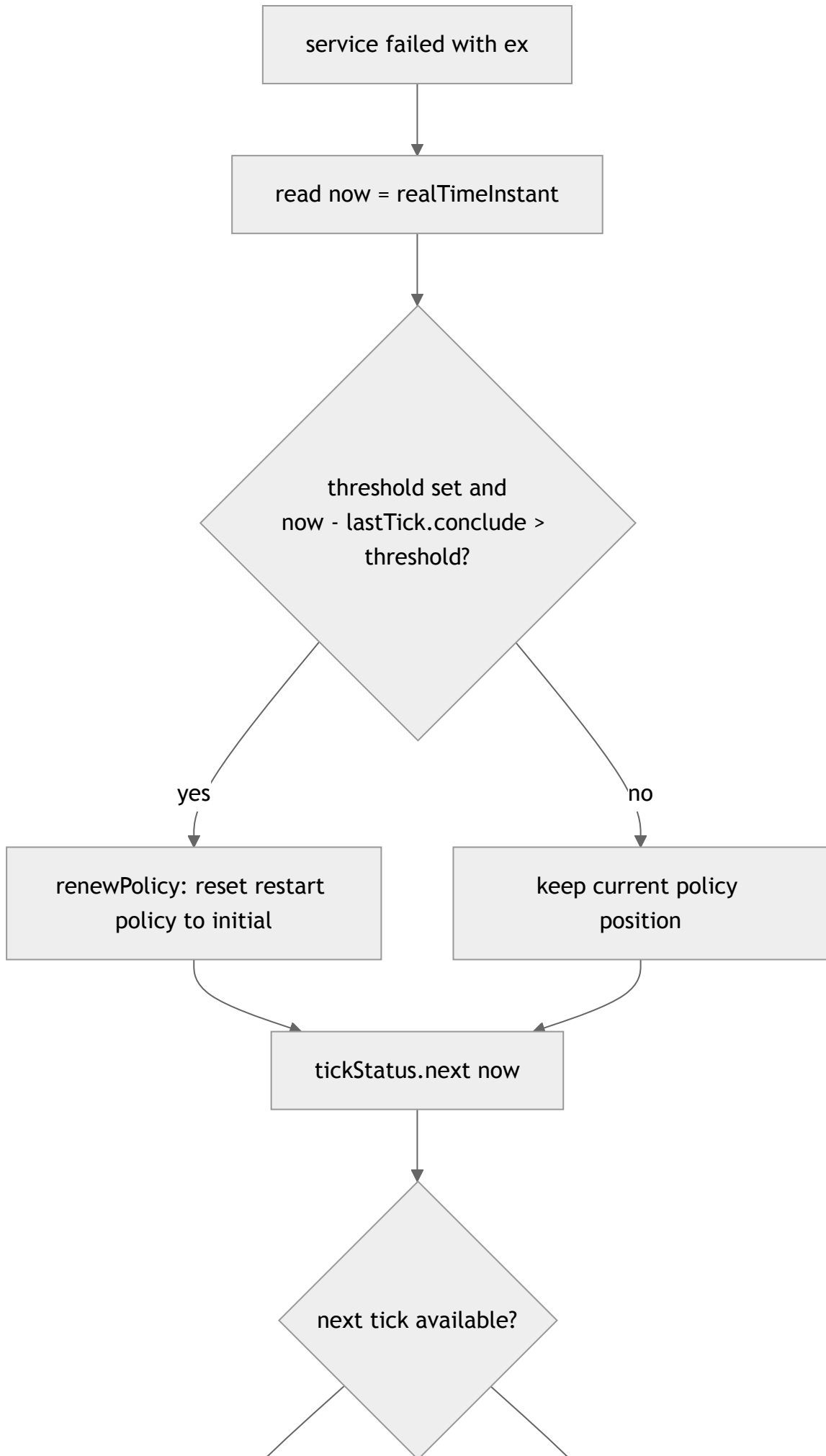
The loop is an `unfoldEval` over a `PolicyTick` (the restart policy's current position). Each iteration starts the service, then branches on how the service effect finished.

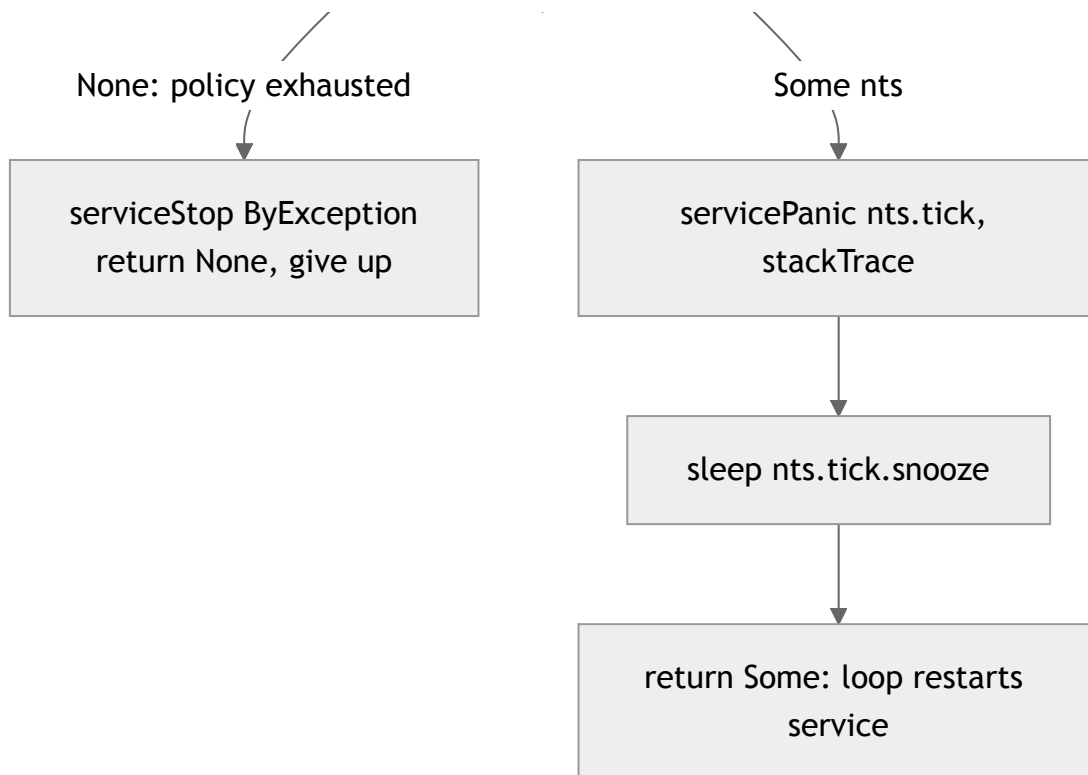


- **success**: the service finished on its own; the watchdog reports `Successfully` and the loop ends. No restart.
- **canceled**: on cancellation `serviceCancel` reports `ByCancellation`, but only if the event channel is not already closed (the check is best-effort and relies on idempotent channel close).
- **error**: control passes to `panic`, which decides restart vs give-up.

Panic: restart or give up

`panic` is where the restart policy is consulted. Two things happen: an optional policy reset based on the success threshold, then a step of the policy to get the next delay.





- **Threshold reset:** `RestartPolicy` carries an optional `threshold`. If the service ran longer than `threshold` since the previous tick concluded, the failure is treated as isolated and the policy is renewed to its initial state (so a long-healthy service that trips gets the full restart budget again). With no threshold, the policy position is preserved across failures.
- **Policy step:** `PolicyTick.next(now)` advances the policy. `None` means the policy is exhausted — the watchdog reports `ByException` and stops for good. `Some(nts)` yields the next tick; the watchdog emits `ServicePanic`, sleeps for that tick's `snooze` delay, then loops to restart the service.

Events emitted

Situation	Event
each (re)start of the service	<code>ServiceStart</code>
a failure that will be retried	<code>ServicePanic</code>
clean completion	<code>ServiceStop(Successfully)</code>
policy exhausted after failure	<code>ServiceStop(ByException)</code>
cancellation	<code>ServiceStop(ByCancellation)</code>

See `../src/main/scala/com/github/chenharryhua/nanjin/guard/service/watchdog.scala` for the loop, `RestartPolicy` in `config/ServiceParams.scala` for the policy/threshold pair, and `PolicyTick` in `common/chrono` for tick stepping and `snooze`.